Services, Singletons & Pipes

shared code, one shared instance, and formatting data

DR
Dinesh Rawat
Instructor · Shorter Loop × Snapied

Today: what an Angular **service** is, how a component gets one through **dependency injection**, and the big idea of a **singleton** (one shared instance) explained with three everyday examples. Then **pipes**, both built-in and your own. We also answer a fair question: why are we learning Angular 13 and not 22?

1 What is an Angular service?

A **service** is a plain class that holds logic or data you want to share across components. Fetching from an API, storing the logged-in user, holding a company name many components show: this belongs in a service, not copied inside every component.

In one line: components are for the view, services are for the shared work. Keep components light, put reusable logic in services.

A service is a class marked with `@Injectable`. The CLI makes one with `ng generate service logo`:

LOGO.SERVICE.TS

```
import { Injectable } from "@angular/core";

@Injectable({ providedIn: "root" })
export class LogoService {
  companyName = "Resume Loop";

  getCompanyName() { return this.companyName; }
  setCompanyName(name: string) { this.companyName = name; }
}
```

`@Injectable` marks the class as injectable. `providedIn: "root"` makes it available across the whole app.

2 Dependency injection: how a component gets a service

A component does not build the service itself. It **asks** for it in the constructor, and Angular **hands it over**. This is **dependency injection** (DI).

HEADER.COMPONENT.TS

```
import { Component } from "@angular/core";
import { LogoService } from "./logo.service";

@Component({
  selector: "app-header",
  template: `
    <h1>{{ logoService.getCompanyName() }}</h1>
    <button (click)="logoService.setCompanyName('Snapied')">Change</button>
  `,
})
export class HeaderComponent {
  constructor(public logoService: LogoService) {}
}
```

The constructor line `public logoService: LogoService` says "I need a LogoService." Angular creates it (or reuses the existing one) and passes it in. You never write `new LogoService()` yourself.

Simple way to picture DI: you do not build your own electricity generator at home. You just plug into the socket, and the supply is provided to you. DI is the same: the component plugs in and asks, Angular provides.

3 Singleton: one shared instance

When a service is `providedIn: "root"`, Angular creates **exactly one** of it for the whole app, and gives that same one to everybody who asks. One shared instance is called a **singleton**.

Why it matters: because everyone shares the same one, if one component changes a value, every other component sees the change. Here are three everyday ways to feel it.

Example 1: The TV remote in a family

There is just **one** TV remote, shared by the whole family. Papa changes the channel to cricket, everyone sees cricket. Little sister picks up the **same** remote and puts on cartoons, now everyone sees cartoons. Nobody has a secret second remote. One remote, one TV, everyone sees the same thing. The service is that one remote.

Example 2: The washing machine in a small family

A small family has **one** washing machine. Everybody shares it. When mother puts a load in and starts it, and later the son comes to use it, he uses the **same** machine, in whatever state it was left. There are not separate machines appearing for each person. One machine, shared by all. The service is that one washing machine.

Example 3: Items and the cart in a mall

You walk around a mall with **one** shopping cart. You drop a shirt in it in one shop, shoes in another, a toy in a third. At billing, the cart still holds everything you added, from every shop. It is the **same** cart the whole time, carrying your shared list across the mall. A singleton service is that one cart: different components add to it and read from it, and it is always the same one.

The opposite, so it is crystal clear: if everyone had their **own** remote, their **own** washing machine, their **own** cart, nothing would be shared. Change one, and the others would know nothing. That is what it would be like **without** a singleton: separate copies, no sharing. The singleton is what makes it one shared thing.

4 Sharing data through the singleton

This is the payoff. Two components that are not parent and child can still share data through the service, because they both get the same singleton.

```
// Header changes it:
this.logoService.setCompanyName("Snapied");

// Footer reads it (in its template):
<p>{{ logoService.getCompanyName() }}</p>   // shows "Snapied"
```

One common mistake: if a component copies the value into its own property once in `ngOnInit`, it will show the **old** value forever, because it read it only once. Read it in the template with `logoService.getCompanyName()` so it always shows the latest. (Think: do not photograph the whiteboard once, keep looking at the board.)

5 Why Angular 13, not 22?

A fair question: the newest Angular is version 22, so why learn 13? A few honest reasons.

- **The core ideas are the same.** Components, services, dependency injection, pipes, modules: these work the same way across versions. What you learn on 13 carries straight over.
- **Most real jobs run older versions.** Companies do not upgrade every year. A huge number of live Angular apps you will work on are on older versions, so knowing them is a real advantage, not a disadvantage.
- **13 is stable and well documented.** Years of tutorials, answers, and examples exist for it, so when you get stuck, help is easy to find.
- **Fewer moving parts for learning.** Newer versions add things like standalone components and signals. Those are powerful, but for a first course they add confusion. Learn the solid base first, then the new features are an easy step.

The takeaway: we learn the strong fundamentals on a stable version. Moving to Angular 22 later is a small jump once the basics are solid. Do not worry that 13 is "old", the thinking is exactly the same.

6 Pipes: format a value for display

A **pipe** changes how a value looks in the template, without changing the real data. You use it with the `|` symbol.

```
<p>{{ name | uppercase }}</p>          <!-- RESUME LOOP -->
<p>{{ price | currency:'INR' }}</p>    <!-- Rs 500.00 -->
<p>{{ today | date:'longDate' }}</p>   <!-- August 7, 2026 -->
```

Common built-in pipes: `uppercase`, `lowercase`, `titlecase`, `date`, `currency`, `number`, `percent`, `json`, `slice`. You can chain them: `{{ name | slice:0:5 | uppercase }}`.

7 Custom pipes: make your own

When no built-in pipe does what you need, write one. A pipe is a class with the `@Pipe` decorator and a `transform` method that takes a value and returns the changed value.

DOUBLE.PIPE.TS

```
import { Pipe, PipeTransform } from "@angular/core";

@Pipe({ name: "double" })
export class DoublePipe implements PipeTransform {
  transform(value: number): number {
    return value * 2;
  }
}
```

USING IT

```
<p>{{ 10 | double }}</p>   <!-- shows 20 -->
```

The `transform` method gets the value on the left of the `|`, does its work, returns the result. Here `10` comes in, `20` goes out. Make one with the CLI: `ng generate pipe double`. Remember to add it to your module's `declarations` (in Angular 13).

A pipe can take an argument, written after a colon:

```
@Pipe({ name: "multiply" })
export class MultiplyPipe implements PipeTransform {
  transform(value: number, times: number): number {
    return value * times;
  }
}

<p>{{ 10 | multiply:3 }}</p>   <!-- shows 30 -->
```

8 Remember this

Service: a class for shared logic and data. `@Injectable` with `providedIn: "root"`.

Dependency injection: ask for a service in the constructor, Angular provides it. Never write `new`.

Singleton: one shared instance for the whole app, like one TV remote, one washing machine, one mall cart. Change it in one place, everyone sees it.

Why Angular 13: the core ideas are identical across versions, most jobs use older versions, and a stable base is easier to learn on.

Pipes: format a value in the template with `|`. Built-in ones like `uppercase` and `date`, or write your own with `@Pipe` and `transform`.

9 Homework

Make the LogoService with `ng g s logo`, inject it into a header component, and show the company name.

Add a button that changes the name through the service, and watch the heading update.

Prove the singleton: read the same service in a second component (a footer). Change the name in the header, confirm the footer shows the new name too.

Write a custom pipe that doubles a number, and use two built-in pipes on some text.